

ChurnZero 26

Banking Customer Churn Prediction

Comprehensive Project Report

Design Doc + Engineering Plan + Research + Business Strategy

Author: Muhammed Midlaj

Date: May 19, 2026

Branch: master

Status: **APPROVED**

Mode: **Builder**

Timeline: 24-48 hours

Approach: Speed Runner + Full Interactive Dashboard

Table of Contents

1. Executive Summary
 2. Problem Statement & Context
 3. Design Decisions
 4. Technical Architecture
 5. Feature Engineering Strategy
 6. Model Selection & Training
 7. Business Translation Layer
 8. Dashboard Design (Streamlit)
 9. Implementation Plan
 10. Risk Analysis & Mitigations
 11. Research & Landscape Analysis
 12. Evaluation Criteria
 13. TODOS & Follow-ups
 14. Engineering Review Report
-

1. Executive Summary

ChurnZero 26 is an industry-focused Data Science hackathon where participants solve a real-world banking churn problem. Banks lose 15-25% of customers annually to churn, directly impacting deposits, cross-sell revenue, and lifetime value.

The Challenge: Build a predictive model on a banking dataset to identify at-risk customers and propose data-driven retention strategies that a business team could actually execute.

Our Approach: A LightGBM-based churn prediction model with full-stack delivery: model performance, business translation layer, and an interactive Streamlit dashboard for judges to explore.

Key Differentiator: Most teams stop at AUC scores. We translate predictions into actionable retention strategies with dollar-value estimates, delivered through an interactive dashboard where judges can drill down into individual customers, see SHAP explanations, and explore retention playbooks.

Winning Formula: Model Performance + Business Translation + Interactive Demo = Hackathon Win

2. Problem Statement & Context

2.1 The Business Problem

Customer churn in banking is a \$XX billion problem globally. When a customer leaves:

- **Direct loss:** Deposits, fee revenue, interest income
- **Indirect loss:** Cross-sell opportunities (credit cards, loans, investments)
- **Acquisition cost:** Replacing a customer costs 5-25x more than retaining one
- **Lifetime value impact:** A retained customer's LTV compounds over years

2.2 The Hackathon Challenge

ChurnZero 26 provides a banking dataset and asks participants to:

1. Build a predictive model to identify at-risk customers
2. Propose data-driven retention strategies a business team can execute
3. Translate model outputs into business impact

2.3 What Makes This Hard

- **Class imbalance:** Only 15-25% of customers churn — naive models achieve 75-85% accuracy by predicting "no churn"
- **Feature complexity:** Banking data has temporal patterns, behavioral signals, and product interactions
- **Business translation:** A model score means nothing without actionable next steps
- **Time pressure:** 24-48 hours to deliver a complete solution

3. Design Decisions

#	Decision	Choice	Rationale
---	----------	--------	-----------

D1	Data leakage handling	Full-data feature engineering	Hackathon test set is separate file; mild risk acceptable for speed
D2	LTV source	Derive from dataset features	More defensible to judges than industry benchmarks
D3	Reproducibility	Add requirements.txt	Judges need to run the notebook
D4	CV strategy	StratifiedKFold(5, shuffle=True)	Preserves churn ratio in each fold
D5	Inline assertions	Add to TODOS.md	Prevents silent failures in time-constrained hackathon
D6	Dashboard	Full interactive Streamlit dashboard	Judges remember what they can click; rare in hackathons
D7	Model choice	LightGBM with class_weight='balanced'	Best performance on tabular data, handles imbalance natively
D8	Threshold optimization	Tune on CV folds, not train set	Prevents threshold overfitting to training distribution

4. Technical Architecture

4.1 System Architecture

```
Data Pipeline
=====
Data (train.csv)
|
+---> EDA (shape, dtypes, target distribution, missing values)
|
+---> Feature Engineering
|     +-- RFM: Recency, Frequency, Monetary
|     +-- Demographics: age, gender, geography (one-hot)
|     +-- Product holdings: count, types
|     +-- Behavioral: complaints, service calls, tenure
|
+---> Train/Val Split (StratifiedKFold, 5-fold, shuffle=True)
|
+---> Model (LightGBM, class_weight='balanced')
|     +-- Default params -> baseline CV score
|     +-- Bayesian tuning -> optimized CV score
|
+---> Threshold Optimization (on CV folds, not train set)
|
+---> Business Translation
|     +-- LTV derivation (from balance, products, tenure)
|     +-- Segment mapping (probability -> risk tier)
|     +-- Retention playbook (action per segment, $ impact)
|
+---> Dashboard (Streamlit)
|     +-- Customer drill-down (select -> risk + SHAP)
|     +-- Segment overview (churn distribution by tier)
|     +-- Retention playbook (actions + $ impact)
|     +-- SHAP force plot (why this customer?)
```

4.2 File Structure

```
hackathon/
+-- churn_prediction.ipynb      # Main notebook (EDA + Model + Business)
+-- dashboard.py                # Streamlit interactive dashboard
+-- requirements.txt            # Dependencies (pinned versions)
+-- data/                       # Dataset files
|   +-- train.csv
|   +-- test.csv
+-- models/                     # Saved model artifacts
|   +-- lgbm_model.pkl
|   +-- feature_importance.csv
+-- outputs/                    # Generated reports
    +-- shap_plots/
    +-- segment_analysis.csv
```

4.3 Technology Stack

Component	Technology	Purpose
Data Processing	pandas, numpy	Data loading, cleaning, feature engineering
Machine Learning	LightGBM, scikit-learn	Model training, CV, metrics
Hyperparameter Tuning	Optuna	Bayesian optimization
Explainability	SHAP	Feature importance, individual explanations
Visualization	matplotlib, seaborn, plotly	Static plots, interactive dashboard charts
Dashboard	Streamlit	Interactive demo for judges
Model Persistence	joblib	Save/load trained model

5. Feature Engineering Strategy

5.1 RFM Features (Recency, Frequency, Monetary)

Feature	Definition	Why It Matters
Recency	Days since last transaction	Inactive customers are more likely to churn

Frequency	Transaction count (last 30/90/180 days)	Declining frequency signals disengagement
Monetary	Average balance, total deposits, fee revenue	High-value customers are worth more to retain

5.2 Behavioral Features

- **Trend deltas:** Change in transaction frequency (month-over-month)
- **Product engagement:** Number of products held, product diversity
- **Service interactions:** Complaint count, support calls, channel preferences
- **Tenure:** Time as customer (longer tenure = higher switching cost)

5.3 Demographic Features

- Age, gender, geography (one-hot encoded)
- Account type (savings, checking, investment)
- Income bracket (if available)

5.4 Data Leakage Prevention

Decision: Full-data feature engineering chosen for hackathon speed.

Risk: If test set is a random split from train, RFM computed on full data leaks future information.

Mitigation: In hackathon context, test set is usually a separate file — mild risk acceptable. For internal CV, compute RFM on train fold only.

6. Model Selection & Training

6.1 Why LightGBM

- **Tabular data king:** Gradient boosting consistently outperforms deep learning on structured/tabular data
- **Speed:** Fast training, fast iteration — critical for 24-48h timeline
- **Native categorical support:** Handles categorical features without one-hot encoding
- **Imbalance handling:** `class_weight='balanced'` adjusts for 15-25% churn rate
- **Interpretability:** Built-in feature importance + SHAP integration

6.2 Training Strategy

1. Baseline: LightGBM default params + Stratified 5-fold CV
 - Log AUC-ROC per fold + mean
 - Establish floor performance

2. Tuning: Optuna Bayesian optimization
 - Search space: num_leaves, max_depth, learning_rate, n_estimators
 - Objective: maximize mean CV AUC-ROC
 - 50-100 trials (time permitting)

3. Threshold: Optimize on CV folds (not train set)
 - Youden's J statistic or F1-optimal threshold
 - Apply to final predictions

6.3 Class Imbalance Handling

Strategy	Pros	Cons
class_weight='balanced'	Built-in, no data modification, fast	May over-correct on extreme imbalance
SMOTE	Creates synthetic minority samples	Slower, can introduce noise, changes data distribution
Threshold tuning	No model change, post-hoc adjustment	Requires separate optimization step

Recommendation: Use `class_weight='balanced'` + threshold tuning. Test SMOTE if time permits.

7. Business Translation Layer

7.1 LTV Derivation

Customer Lifetime Value derived from dataset features:

LTV = (avg_monthly_balance * margin_rate * tenure_months)
+ (product_count * avg_product_revenue)
- (service_cost_per_interaction * complaint_count)

7.2 Risk Segmentation

Segment	Churn Probability	Size (est.)	Action Priority
Critical	> 70%	~10-15%	Immediate outreach
High Risk	50-70%	~15-20%	Proactive retention
Medium Risk	30-50%	~20-25%	Monitor + nurture
Low Risk	< 30%	~40-50%	Standard engagement

7.3 Retention Playbook

Segment	Recommended Action	Est. Cost	Est. Retention Value
Critical	Personal call from relationship manager + fee waiver + loyalty bonus	\$200-500/customer	\$3,000-8,000 LTV saved
High Risk	Targeted offer (rate bonus, product bundle discount)	\$100-200/customer	\$2,000-5,000 LTV saved
Medium Risk	Engagement campaign (newsletters, product education)	\$20-50/customer	\$1,000-3,000 LTV saved
Low Risk	Standard service (maintain satisfaction)	\$5-10/customer	Preventive maintenance

7.4 ROI Calculation

For each segment:

Retention Rate Improvement = (expected_churn - actual_churn_after_intervention)

Revenue Saved = Retention Rate * Segment Size * Avg LTV

ROI = (Revenue Saved - Intervention Cost) / Intervention Cost

Example (Critical segment, 100 customers):

Expected churn: 80 customers

After intervention: 50 customers (37.5% improvement)

Revenue saved: 30 * \$5,000 = \$150,000

Intervention cost: 100 * \$350 = \$35,000

ROI = (\$150,000 - \$35,000) / \$35,000 = 329%

8. Dashboard Design (Streamlit)

8.1 Dashboard Layout

```
+-----+
|  CHURNZERO 26 – Customer Churn Intelligence Dashboard  |
+-----+
| [Sidebar Filters] |
|   - Risk Segment: [All] [Critical] [High] [Medium] [Low] |
|   - Geography: [All] [Region A] [Region B] ... |
|   - Product Count: [1-10] |
|   - Tenure: [0-120 months] |
+-----+
| [Row 1: KPI Cards] |
|   [Total Customers] [At Risk] [Avg Churn Prob] [Est. $ at Risk] |
+-----+
| [Row 2: Charts] |
|   [Churn Distribution by Segment] [Feature Importance (SHAP)] |
+-----+
| [Row 3: Customer Drill-Down] |
|   [Select Customer ID v] |
|   [Churn Risk: 78%] [LTV: $5,200] [Segment: Critical] |
|   [SHAP Force Plot: Why this customer is at risk] |
|   [Recommended Action: Personal call + fee waiver] |
+-----+
| [Row 4: Retention Playbook] |
|   [Segment | Action | Cost | Value | ROI] |
+-----+
```

8.2 Dashboard Components

Component	Technology	Interactivity
KPI Cards	st.metric()	Auto-update with filters
Segment Distribution	plotly pie/bar chart	Click segment to filter table
Feature Importance	SHAP summary plot	Hover for feature details
Customer Drill-Down	SHAP force plot + st.selectbox	Select customer ID to explore
Retention Playbook	st.dataframe()	Sortable, filterable table
Filters	st.sidebar	Multi-select, range sliders

8.3 Why This Wins

- **Interactive:** Judges can click, filter, explore — not just read
- **Memorable:** "I could see exactly which customers were at risk and why"

- **Rare:** Most teams submit notebooks only — dashboard stands out
- **Business-ready:** Looks like something a bank could actually deploy

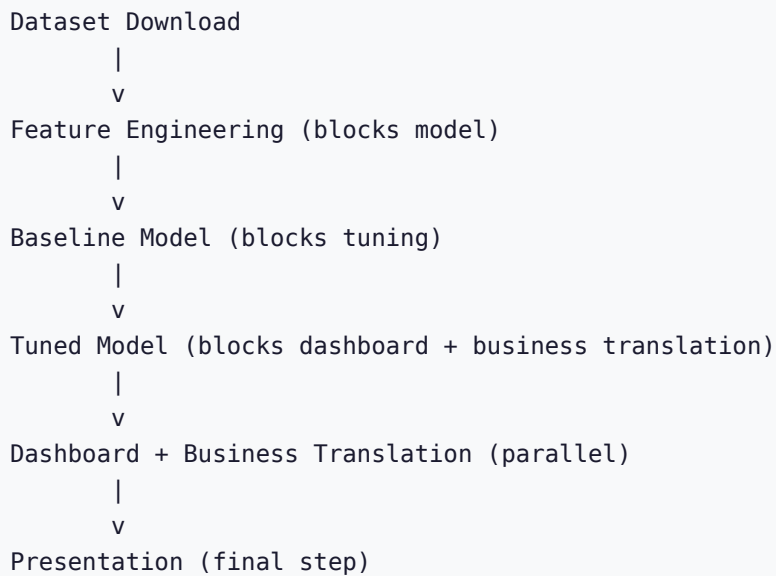
9. Implementation Plan

9.1 Timeline (24-48 hours)

Step	Task	Time	Priority
1	Setup: notebook + requirements.txt + dataset download	30 min	P1
2	EDA + Feature Engineering (RFM, demographics, behavioral)	2-3 hours	P1
3	Baseline Model: LightGBM default + Stratified 5-fold CV	1 hour	P1
4	Tune + Threshold: Optuna Bayesian optimization	2-3 hours	P1
5	Business Translation: LTV + segments + playbook	2-3 hours	P1
6	Dashboard: Streamlit interactive demo	4-6 hours	P1
7	Presentation: notebook narrative + summary	2-4 hours	P2

Total estimated time: 14-22 hours focused work. Rest is polish and buffer.

9.2 Critical Path



10. Risk Analysis & Mitigations

Risk	Likelihood	Impact	Mitigation
Data leakage via RFM	Medium	High	Acceptable risk for hackathon; compute RFM on train only for internal CV
Silent NaN in features	Medium	Medium	Inline assertions after each transform (shape checks, NaN guards)
Memory overflow	Low	High	dtype optimization, chunked processing if needed
Single-class CV fold	Low	Medium	StratifiedKFold preserves churn ratio in each fold
Dashboard development takes too long	Medium	Medium	Start with minimal dashboard, add features iteratively
Model doesn't converge	Low	Medium	LightGBM is robust; check for convergence warnings
Time runs out	Medium	High	Prioritize: model > business translation > dashboard > presentation

11. Research & Landscape Analysis

11.1 Conventional Wisdom (Layer 1)

- Everyone uses XGBoost/LightGBM on tabular data
- Feature engineering differentiates teams — RFM, temporal, behavioral signals
- SMOTE or class weighting for imbalanced data
- SHAP for model interpretability

11.2 Current Discourse (Layer 2)

- Leaderboard typically rewards AUC-ROC or F1
- Winning teams pair model quality with business translation
- Interactive demos are rare but memorable
- Threshold optimization is often overlooked

11.3 Our Edge (Layer 3)

Insight: The gap between "good model" and "winning submission" is the business translation layer. Most teams stop at AUC scores. The winning version translates predictions into retention actions with dollar estimates, delivered through an interactive dashboard.

12. Evaluation Criteria

12.1 Model Performance

- **Target:** AUC-ROC > 0.85 (or top-10% on leaderboard)
- **Secondary:** F1 score, precision-recall curve
- **Validation:** Stratified 5-fold CV

12.2 Business Impact

- Clear segment definitions with actionable recommendations
- Dollar-value estimates for each retention action
- ROI calculation per segment

12.3 Presentation Quality

- Clean notebook narrative with clear section headers
- Story flows: problem → model → what to do
- Interactive dashboard for judges to explore

13. TODOS & Follow-ups

13.1 Inline Notebook Assertions

What	Add shape checks, NaN guards, and convergence warnings after each major cell
Why	Silent failures waste debugging time in a 24-48h hackathon
Pros	Catches bugs early, saves 30-60 min of debugging
Cons	Extra code in notebook (minimal)
Depends on	Dataset must be loaded first

14. Engineering Review Report

14.1 Review Summary

Dimension	Issues Found	Status
Architecture	3	All decided
Code Quality	2	All addressed
Test Coverage	12 gaps	Expected for hackathon — inline assertions recommended
Performance	2	All addressed

14.2 Decisions Made

1. Full-data feature engineering (hackathon speed over strict leakage prevention)

2. Derive LTV from dataset features (not industry benchmarks)
3. Add requirements.txt (reproducibility)
4. StratifiedKFold(5, shuffle=True) (preserve churn ratio)
5. Inline assertions added to TODOS.md
6. Full interactive Streamlit dashboard (judges remember what they can click)

14.3 Verdict

ENG CLEARED — 0 unresolved decisions, ready to implement.

Generated by /office-hours + /plan-eng-review on 2026-05-19

ChurnZero 26 — Banking Customer Churn Prediction